CSCI-739 Quantum Machine Learning

Homework 4

You made it, team. This homework moves from expressivity to practicality: from the theoretical foundations of quantum neural networks to their real implementation limits. You will see how quantum encodings shape the functions a model can represent (P1), how depth affects performance in classical and quantum learning (P2), and, through the bonuses, how hardware, state complexity, and simulation cost bound what is possible on current machines. This time you also get to choose your path: work through the problems, or take on a small research project on MNIST. Each problem is a lens on scalability, trainability, and the physics of information. You got this. ☺

Choose your path (this homework is special). Full credit (100 pts) can be earned in either of two ways:

- **Path 1 — Problems:** Problems 1 and 2 (50 pts each).
- **Path 2 — MNIST project:** the quantum MNIST project (100 pts), where you study encodings, what is and is not doable, and PCA-compressed versus full-input models.

You may also do *both* paths, and the bonus problems are open to everyone. The points add up:

Component	Points
Problem 1 — Expressivity and universal approximation	50
Problem 2 — Classical vs. quantum networks on a sine	50
MNIST project — Encodings, PCA, and VQC/QNN classifiers	100
Bonus A — Diagonal Hamiltonian and VQE scaling	15
Bonus B — Complexity of quantum state preparation	15
Bonus C — Multivariate sine regression	20
Everything	250
Typeset in LaTeX	+10
Maximum possible	**260**

Structure overview:

- *P1:* Theoretical expressivity and the universal approximation property of QNNs.
- *P2:* Empirical study of depth and performance in classical vs. quantum neural networks.
- *MNIST project:* Encodings, resource limits, and PCA-compressed vs. full-input quantum and classical classifiers.
- *Bonus A:* The "spooky" diagonal Hamiltonian: analytical vs. VQE ground-state scaling.

- *Bonus B:* The complexity hierarchy of quantum state preparation.
- *Bonus C:* Multivariate sine regression under increasing input dimension and depth.

Use lectures, notes, the internet, and each other to research the problems and conquer this homework, team! Have fun ☺! Also:



Problem 1 — Expressivity and Universal Approximation in Quantum Neural Networks (50 pts)

Summary. We study how data encodings and parameterized unitaries determine the expressivity of quantum neural networks (QNNs). You will derive conditions under which a QNN can approx-

imate nonlinear functions of classical inputs, compare encoding choices, and connect these results to the universal approximation theorem for classical neural networks.

A general QNN defines the function

$$f_{\mathrm{QNN}}(x;\boldsymbol{\theta}) \;=\; \langle 0|^{\otimes n}\, U_\phi(x)^\dagger\, U(\boldsymbol{\theta})^\dagger\, M\, U(\boldsymbol{\theta})\, U_\phi(x)\, |0\rangle^{\otimes n},$$

where

- $x \in \mathbb{R}^d$ is the classical input vector,
- $U_\phi(x)$ is the **encoding unitary** (basis, angle, or amplitude encoding),
- $U(\boldsymbol{\theta})$ is the **trainable ansatz** of parameterized gates,
- M is the measurement observable (often Z or Z_i),
- and $f_{\mathrm{QNN}}(x;\boldsymbol{\theta}) \in [-1,1]$ is the predicted value (for M with eigenvalues ± 1).

(a) **Encoding and expressivity.** Compare basis encoding, which maps a bitstring $x \in \{0,1\}^n$ to the computational-basis state $|x\rangle$, with angle encoding, which maps a real number x to $R_y(\pi x)\,|0\rangle$. Show that for one-qubit angle encoding, $f_{\mathrm{QNN}}(x;\theta)$ is a trigonometric polynomial in x of the form $a_0 + a_1\cos(\pi x) + b_1\sin(\pi x)$, with coefficients that depend on θ. Discuss why this smooth nonlinearity increases representational capacity relative to basis encoding, where the model can only assign values to the 2^n discrete inputs.

(b) **Simple one-qubit example.** Let $U_\phi(x) = R_y(\pi x)$, $U(\theta) = R_y(\theta)$, and $M = Z$. Compute

$$f_{\mathrm{QNN}}(x;\theta) = \langle 0|\, R_y(-\pi x) R_y(-\theta)\, Z\, R_y(\theta) R_y(\pi x)\, |0\rangle\,.$$

Simplify analytically and describe the functional form of $f_{\mathrm{QNN}}(x;\theta)$. Which functions of x can this model represent, and which can it not?

(c) **Universal approximation.** Discuss qualitatively how multi-qubit, layered QNNs can approximate arbitrary continuous functions $f : [0,1]^d \to [-1,1]$, analogous to classical feedforward networks. Relate this to the idea that as the number of encoding gates (through more qubits or by re-uploading the data in each layer) grows, the set of realizable trigonometric polynomials becomes dense in $C([0,1]^d)$.

(d) **Relationship between classical and quantum NNs.** Suppose each feature enters the circuit only once, through a single rotation angle, and the ansatz is a single shallow layer. Show that f_{QNN} is then a linear combination of a small, fixed set of features such as $\{1, \cos(\pi x_k), \sin(\pi x_k)\}$ (and their products, if the layer entangles qubits). Explain why such a model is no more powerful than a classical linear model on those features, and what has to change (depth, re-uploading, entanglement) to go beyond it.

Problem 2 — Classical vs. Quantum Neural Networks on a Sine Function (50 pts)

Summary. You will implement a classical and a quantum neural network to learn a one-dimensional sine function. By varying the number of layers, you will observe when additional depth helps and when it does not, illustrating the interplay between expressivity and trainability. You will also see that *how* a quantum model is layered matters as much as how many layers it has.

Target function.

$$y(x) = \sin(2\pi x), \qquad x \in [0, 1].$$

Dataset. Use $N_{\text{train}} = 64$ uniformly spaced training points and $N_{\text{test}} = 256$ uniformly spaced test points.

Objective (common to both models).

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \big(\hat{y}(x_i; \boldsymbol{\theta}) - y(x_i)\big)^2,$$

where $\hat{y}$ is the output of the model (classical or quantum).

Classical model (MLP baseline)

Use a multilayer perceptron with 1 input, 1 output, and hidden layers of width ≥ 16 with tanh activation. Train for $L \in \{1, 2, 4, 8\}$ hidden layers using Adam or another optimizer of your choice (keep it fixed across L). Plot train/test MSE vs. epochs and compare performance across L.

Quantum model (QNN)

Encoding. Angle encoding, as in Problem 1:

$$U_\phi(x) = R_y(\pi x).$$

(i) Warm-up: a stacked R_y ansatz. Consider first the ansatz used in Problem 1, stacked L times,

$$U(\theta_1, \ldots, \theta_L) = R_y(\theta_L) \cdots R_y(\theta_1).$$

Show that this model gives exactly the same function class for every L, and find that class. Can it fit $\sin(2\pi x)$ for any L? Why not? (Hint: do rotations about the same axis commute? What frequencies in x can appear?)

(ii) Data re-uploading ansatz (train this one). To make depth matter, re-upload the data in every layer and use rotations about two different axes:

$$U_L(x; \boldsymbol{\theta}) = \prod_{\ell=1}^{L} \Big[R_y(\theta_{\ell,1})\, R_z(\theta_{\ell,2})\, R_y(\pi x) \Big] \qquad \text{(rightmost factor, } \ell = 1, \text{ acts first).}$$

Model output.

$$f_{\mathrm{QNN}}(x;\boldsymbol{\theta}) = \langle 0|\, U_L(x;\boldsymbol{\theta})^\dagger\, Z\, U_L(x;\boldsymbol{\theta})\,|0\rangle\,.$$

Estimate $\langle Z\rangle$ by shot-based sampling with S shots, for example $S = 1000$, using $\widehat{\langle Z\rangle} = (n_0 - n_1)/S$, where n_0 and n_1 count the outcomes 0 and 1. Compare against an exact statevector simulation (no shots).

Optimization. For a gate of the form $e^{-i\theta P/2}$ with P a Pauli operator (such as R_y and R_z), the parameter-shift rule gives the exact derivative of the *expectation value*:

$$\frac{\partial f_{\mathrm{QNN}}}{\partial\theta} = \frac{1}{2}\Big[f_{\mathrm{QNN}}\big(\theta + \tfrac{\pi}{2}\big) - f_{\mathrm{QNN}}\big(\theta - \tfrac{\pi}{2}\big)\Big].$$

The rule applies to f_{QNN}, not directly to the loss. Use the chain rule to get the gradient of the MSE:

$$\frac{\partial\mathcal{L}}{\partial\theta} = \frac{2}{N}\sum_{i=1}^{N}\big(f_{\mathrm{QNN}}(x_i;\boldsymbol{\theta}) - y(x_i)\big)\,\frac{\partial f_{\mathrm{QNN}}(x_i;\boldsymbol{\theta})}{\partial\theta}.$$

Experiments. For $L \in \{1, 2, 4, 8\}$:

- Train both the MLP and the re-uploading QNN on the same data.
- Compare MSE and convergence behavior.
- What is the smallest L for which the QNN can represent $\sin(2\pi x)$ exactly? Relate this to the frequencies in x that L encoding gates can produce.
- Discuss why depth helps at first, and why it may slow training or shrink gradients as L grows further.

Deliverables. For both the classical and quantum models: plots of predicted vs. true $\sin(2\pi x)$, loss vs. epochs, and gradient norm vs. epochs, for each L. Include your answer to the warm-up (i). Notice anything interesting? ☺

Path 2 — Quantum MNIST Project: Encodings, PCA, and What Is (and Is Not) Doable (100 pts)

Summary. This project replaces Problems 1 and 2 for full credit (or adds to them, if you do both). You will build variational quantum circuit (VQC) / quantum neural network (QNN) classifiers for handwritten digits from MNIST. A 28×28 image has 784 pixels, far more than any qubit budget we can simulate with a naive encoding. You will therefore (i) study which encodings are possible and what each one costs, (ii) compress the images with principal component analysis (PCA) and train QNNs on the low-dimensional data, and (iii) compare these compressed models with models that see the full input. The goal is not a record accuracy; it is an honest, quantitative picture of what quantum models can and cannot do with real data today.

Data. Load MNIST, for example with `sklearn.datasets.fetch_openml('mnist_784')` or `torchvision.datasets.MNIST`. Start with a binary task:

- an easy pair, **0 vs. 1**;
- a harder pair of your choice, e.g. **3 vs. 5** or **4 vs. 9**.

Quantum simulation is slow, so subsample: for example 1000 training and 500 test images per task, balanced between the two classes. Use the same subsets for every model. Scale pixel values to $[0, 1]$.

PCA. Fit PCA on the *training set only* (fitting on the test set leaks information) and keep the top k components, $k \in \{2, 4, 8, 16\}$. Then rescale each component to an angle range such as $[0, \pi]$ using the training-set minimum and maximum.

Why PCA, and what it costs. PCA projects each centered image onto the k orthogonal directions of largest variance in the training set (the top eigenvectors of the sample covariance matrix). Keep these trade-offs in mind as you work; part (f) asks you to weigh them with your own results.

PCA in classical machine learning.

- **Pros:** it reduces dimension sharply (on MNIST, a few dozen components keep most of the pixel variance, which you will measure in (a)); it removes correlated and nearly constant pixels (the image borders) and some noise; it makes models smaller, faster to train, and less prone to overfitting when data are scarce; it gives 2D and 3D pictures of the data; it is cheap, deterministic, and has a closed form.
- **Cons:** it is *linear*, so curved structure in the data (digit shapes under stroke thickness, slant, and shift) is only approximated; it is *unsupervised*, so the directions of largest variance need not be the ones that separate the classes, and a small-variance direction that tells 4 from 9 can be thrown away; components are global mixtures of pixels and hard to interpret; it is sensitive to how features are scaled; and whatever it discards is gone for every model trained afterwards. Modern classical models (CNNs) usually work on the raw pixels and learn their own features instead.

PCA first, then a VQC.

- **Pros:** it is often the only way to fit the data onto the qubits at all (k qubits with angle encoding instead of 784); it keeps the encoding circuit shallow, which matters on noisy NISQ hardware (compare amplitude encoding in (b)); it keeps the number of trainable parameters, the shot budget, and the simulation cost small; and it gives a clean, controlled experiment in which k sets the problem size for every model.
- **Cons:** the classical preprocessing does much of the work, and the quantum model can only use what PCA kept; if a good classifier exists on k features, a small classical model on the same k features usually finds it too, so the fair baseline is a classical model on the *same* PCA features, not on raw pixels; any claim of a quantum advantage then concerns only the low-dimensional problem; the components carry very unequal variance, so the qubits encoding the last components carry little information; the rescaling to angles is a modeling choice (angle encoding is periodic, so values far outside $[0, \pi]$ wrap around); and PCA itself is a classical step that has to be computed on the full data first. (Quantum PCA algorithms exist, but they need quantum access to the data and fault-tolerant hardware.)

(a) **Data, PCA, and classical baselines (15 pts).**

- Plot the cumulative explained variance vs. k, and show a few images reconstructed from $k = 2, 4, 8, 16$ components next to the originals.
- Train two classical baselines, logistic regression and a small MLP, on (1) the full 784-pixel input and (2) the PCA-k features for each k. Report test accuracy and the number of trainable parameters.
- For $k = 2$, make a scatter plot of the two classes in PCA space. Is the easy pair already separable?

(b) **Encodings: what is and is not doable (20 pts).** For each encoding below, state how many qubits it needs and how the circuit depth (or gate count) needed to load one image scales, both for the full 784-pixel image and for PCA-k features. Then state whether you can simulate it on your machine and whether it could run on current NISQ hardware. Summarize in a table.

- **Basis encoding:** binarize the pixels and write them into computational-basis states.
- **Angle encoding:** one feature per qubit, $\bigotimes_k R_y(x_k)$.
- **Dense angle encoding:** two features per qubit, e.g. $R_z(x_{2k})R_y(x_{2k-1})$.
- **Amplitude encoding:** the normalized feature vector becomes the amplitudes of an m-qubit state, $|\psi(x)\rangle = \sum_j \frac{x_j}{\|x\|} |j\rangle$. A full image, zero-padded to $1024 = 2^{10}$ entries, fits on $m = 10$ qubits.
- **Data re-uploading:** angle encoding repeated in every layer, as in Problem 2.

Discuss: a 784-qubit angle-encoded circuit *without* entangling gates is easy to simulate classically (why?), while the same circuit *with* entangling gates is not. Amplitude encoding is qubit-cheap but needs $\mathcal{O}(2^m)$ gates for a general state (connect to Bonus B if you do it). Which entries of your table are impossible, which are possible but impractical, and which are easy?

(c) **QNN on PCA features (25 pts).** Implement a VQC classifier on the PCA-k features for $k \in \{2, 4, 8\}$ (and 16 if your simulator allows it), using angle or dense angle encoding with k or $k/2$ qubits.

- Ansatz: $L \in \{1, 2, 4\}$ layers of single-qubit rotations (R_y, R_z) followed by a ring of CNOTs (a single CNOT when there are only 2 qubits). Re-uploading is encouraged.
- Readout: $\langle Z_1 \rangle$ mapped to a class probability $p = (1 - \langle Z_1 \rangle)/2$, or a trainable linear head on all $\langle Z_k \rangle$ as in Bonus C.
- Loss: binary cross-entropy or MSE; state which one you use.
- Gradients: autodifferentiation through a statevector simulator is fine for training, but verify the parameter-shift gradient against it for at least one model and report the agreement.
- Report train/test accuracy and loss curves for every (k, L), and the gradient norm at initialization vs. the number of qubits and L. Do you see signs of a barren plateau?

(d) **A full-input quantum model (15 pts).** Build a quantum model that sees the *entire* image: amplitude-encode the padded 1024-dimensional image on 10 qubits and train a VQC on top of it. You may use your simulator's direct state initialization; report how many gates a real circuit for this state preparation would need (e.g. by transpiling the `initialize`/`StatePreparation` circuit in Qiskit for a few images). Compare accuracy and cost with the amplitude-encoded PCA-k models ($k = 16$ fits on 4 qubits). Does the extra information in the full image help the quantum model, and what does it cost?

(e) **Comparison (15 pts).** On one figure per task, plot test accuracy vs. input dimension k for: logistic regression, MLP, and your QNN(s), with the full-input classical and quantum results as reference lines. Make a second plot of test accuracy vs. number of trainable parameters. Also report training wall-clock time. Where do the quantum models match the classical ones, and where do they fall behind?

(f) **Report and reflection (10 pts).** Write a report with your plots, tables, and the encoding table from (b). There is no page requirement: use as much or as little space as you need to tell the story. Answer: How much of the digit information survives PCA at small k, and is that why small QNNs can work at all? Using your own results, weigh the pros and cons of PCA listed above: for the classical models, did PCA help or hurt compared with the full 784-pixel input? For the quantum models, how much of the accuracy comes from PCA and how much from the VQC (compare with the classical models on the same k features)? Did the harder digit pair suffer more from PCA than the easy one, and why? Which encodings would you choose for a real quantum device today, and why? What would have to change, in hardware or in algorithms, for a quantum model to use all 784 pixels without PCA? Include your code.

(g) **(Optional, for fun) Going further.** Try the hardest version you can: a 10-class classifier (e.g. one output qubit per class, or one-vs-rest), a run of your smallest trained model on IBM NISQ hardware with $S \in \{10, 100, 1000\}$ shots, or a hybrid model with a classical layer before the quantum circuit. Compare with your simulator results.

Suggested tools. Qiskit or PennyLane for circuits, `scikit-learn` for PCA and the classical baselines, and PyTorch or JAX if you want autodifferentiation. Your own NumPy statevector simulator is also welcome for ≤ 12 qubits.

Bonus A (the spooky question that disappeared from HW3 and returned in HW4) — Ground States of a Purely Diagonal n-Qubit Hamiltonian with an Indexed Spectrum (15 pts)

Summary. We study an n-qubit Hamiltonian that is diagonal in the computational basis and whose diagonal entries increase with the integer index of the basis state. You will (i) derive the spectrum and ground state analytically, (ii) compare the classical storage cost 2^n with the quantum cost n, and (iii) implement a minimal VQE that estimates the ground energy by sampling Z-basis measurements. This problem shows how even a simple diagonal Hamiltonian illustrates the exponential gap between classical and quantum representations.

Definition (diagonal by basis index). Label computational-basis states $|j\rangle$ by $j \in \{0, 1, \ldots, 2^n - 1\}$ using their binary expansion. Define

$$H_n = \mathrm{diag}(E_j)_{j=0}^{2^n-1}, \qquad E_j = j + 1.$$

All off-diagonal elements are zero.

- A Z-basis measurement returns a bitstring

  $$\mathbf{b} = (b_{n-1} \ldots b_1 b_0) \in \{0, 1\}^n,$$

  corresponding to the integer $j = \sum_{i=0}^{n-1} 2^i b_i$.

- The random variable J ("measurement outcome as an integer") has

  $$p_j = \Pr(J = j) = |\langle j | \psi|^2.$$

$$\langle J \rangle = \sum_{j=0}^{2^n-1} j\, p_j, \quad E_n(\boldsymbol{\theta}) = \sum_j (j+1) p_j = \langle J \rangle + 1.$$

For S shots:

$$\widehat{\langle J \rangle} = \frac{1}{S} \sum_{t=1}^{S} j^{(t)}, \qquad \widehat{E}_n = \widehat{\langle J \rangle} + 1, \quad \mathrm{Var}[\widehat{\langle J \rangle}] = \frac{\mathrm{Var}(J)}{S}.$$

(a) **Eigenvalues and eigenstates.** Show that $|j\rangle = e_{j+1}$ (the $(j+1)$-th standard basis vector) is an eigenvector of H_n with eigenvalue $E_j = j + 1$.

(b) **Ground state and energy, and classical computational cost.** Show that $E_0^{(\mathrm{an})}(n) = 1$, with ground state $|0 \cdots 0\rangle = e_1$.

Now reason about classical feasibility: the Hamiltonian is $2^n \times 2^n$, so it has 4^n entries if stored as a dense matrix. For $n = 10$ this is $\sim 10^6$ entries; for $n = 40$ it is $\sim 10^{24}$, impossible even on supercomputers. A molecule such as Fe_2S_2, or a protein fragment mapped to $n \sim 40$ logical qubits, has a $2^{40} \times 2^{40}$ Hamiltonian, far beyond HPC capacity. Explain why an n-qubit quantum device represents these 2^n amplitudes implicitly, and how this motivates VQE.

(c) **Classical vs. quantum scaling.** Plot 2^n and n for $n = 1, \ldots, 40$ on a logarithmic scale. Comment on the classical versus quantum resources needed to represent an n-qubit system. Also comment on the feasibility of finding the lowest energy and the corresponding ground state for $n = 40$ with state-of-the-art numerical eigenvalue solvers. Do we have a problem ☺? How many qubits would you need to approximate the lowest energy and the corresponding ground state on a quantum computer?

(d) **Ansatz (no entanglers).**

$$|\psi(\boldsymbol{\theta})\rangle = \bigotimes_{i=1}^{n} R_y(\theta_i)\,|0\rangle\,.$$

Minimize $\widehat{E}_n = \widehat{\langle J\rangle} + 1$ in two ways: first, find analytically the $\boldsymbol{\theta}^*$ that minimizes the energy; second, minimize it with a numerical optimizer of your choice. Do the two answers agree? Is the answer what you expected?

For context: let $b_i^{(t)} \in \{0,1\}$ denote the measurement outcome of the i-th qubit on the t-th shot, where $i = 0$ corresponds to the **least significant bit (LSB)** of the binary representation of the measured integer $j^{(t)}$. Formally, each n-qubit computational-basis outcome $\mathbf{b} = (b_{n-1}\ldots b_1 b_0)$ corresponds to the integer

$$j^{(t)} = \sum_{i=0}^{n-1} 2^i\, b_i^{(t)},$$

where b_0 is the LSB (the rightmost bit) and b_{n-1} is the most significant bit (MSB).

Define $n_1^{(i)} = \sum_{t=1}^{S} b_i^{(t)}$ as the number of shots in which qubit i is measured in the state $|1\rangle$, and $\hat{p}_i = n_1^{(i)}/S$ as the corresponding empirical probability. Then, because expectation values are linear,

$$\boxed{\widehat{\langle J\rangle} = \sum_{i=0}^{n-1} 2^i\,\hat{p}_i = \sum_{i=0}^{n-1} 2^i\,\frac{n_1^{(i)}}{S}.}$$

This bitwise expression is equivalent to computing

$$\widehat{\langle J\rangle} = \frac{1}{S}\sum_{t=1}^{S} j^{(t)} = \sum_{j=0}^{2^n-1} j\,\frac{c_j}{S},$$

where c_j is the number of times the outcome j is observed among the S shots.

(e) **Comparison across sizes.** Plot $\widehat{E}_n^{(\mathrm{vqe})}$ against $E_0^{(\mathrm{an})}(n) = 1$ and discuss how the variance of the estimator grows with n away from the optimum.

(f) **Resource limits.** Track time and memory vs. n and identify when simulation bottlenecks appear. At what value of n does Qiskit's statevector simulator complain? Did you get to 40 ☺ for this extremely simple ground-state problem, where we already know the answer? Comment on the difficulty of simulating a quantum system on a classical computer in general, and how this motivates the interest in VQCs for finding the lowest energy and the corresponding ground state of a Hamiltonian H.

(g) **(Optional) Real hardware shot sweep.** Run on IBM hardware for $S \in \{1, 10, 100\}$ and check that $\mathrm{Var}[\widehat{\langle J\rangle}] = \mathrm{Var}(J)/S$.

Bonus B — The Complexity of Quantum State Preparation (15 pts)

Summary. We examine how hard it is to prepare quantum states. Every normalized vector exists mathematically, but preparing it physically with a finite circuit can range from trivial to impossible. You will reason through three regimes: (i) easily preparable states, (ii) exponentially hard states, and (iii) states that no physical process can produce. This reveals how state-preparation complexity limits QNN and VQE ansätze. For each example, draw the circuit (where one exists) and give the circuit depth required to prepare the state in big-$\mathcal{O}$ notation as a function of n: polynomial, exponential, or impossible. For impossible states, explain which physical law, rule, or concept forbids them ☺.

(a) **Easily preparable states.**

- **Uniform superposition:**
$$|\psi_{\text{unif}}\rangle = \frac{1}{\sqrt{2^n}} \sum_{j=0}^{2^n-1} |j\rangle \,.$$
- **GHZ state:**
$$|\text{GHZ}_n\rangle = \frac{1}{\sqrt{2}}\big(|0\rangle^{\otimes n} + |1\rangle^{\otimes n} \big).$$
Can you reduce the depth from $\mathcal{O}(n)$ to $\mathcal{O}(\log n)$?

(b) **Computationally hard states.**

- **Haar-random state:**
$$|\psi_{\text{Haar}}\rangle = \sum_{j=0}^{2^n-1} c_j \,|j\rangle \,, \qquad c_j = \frac{g_j}{\sqrt{\sum_k |g_k|^2}}, \quad g_j \sim \mathcal{N}_{\mathbb{C}}(0,1) \text{ i.i.d.}$$
Use a counting argument (how many states can circuits with G gates reach?) to argue that almost all such states need exponentially many gates.
- **Frustrated spin-glass ground state:**
$$H = \sum_{\langle i,j \rangle} J_{ij} Z_i Z_j + \sum_i h_i Z_i,$$
where $Z_i = I \otimes \cdots \otimes I \otimes Z \otimes I \otimes \cdots \otimes I$ is a Z gate on the i-th qubit. This H is diagonal, so its ground state is a computational-basis state. Explain why *preparing* that state is easy once you know it, while *finding* it is hard in general (NP-hard). How does this kind of hardness differ from that of the Haar-random state?

(c) **Impossible states.**

- Non-normalizable states, e.g. a position eigenstate $|x\rangle$.
- No-cloning: you cannot prepare $|\psi\rangle \otimes |\psi\rangle$ from a single copy of an unknown, general state $|\psi\rangle$.
- Complex conjugation: you cannot deterministically map an unknown $|\psi\rangle$ to its complex conjugate $|\psi^*\rangle$. (Hint: is this map linear? Is it unitary?)

Reflection: How does state-preparation complexity constrain ansatz design in NISQ QML? Hint: data encoding *is* state preparation in quantum machine learning, so some encodings are easy, some hard, and some impossible, depending on how you wish to encode your data. Also think about circuit depth and why it is a problem in the NISQ era ☺.

Bonus C — Multivariate Sine Regression with Classical vs. Quantum Neural Networks (20 pts)

Summary. This bonus generalizes Problem 2 from a one-dimensional sine to an n-variable target. You will implement a classical neural network (MLP) and a quantum neural network (QNN) for regression on $x \in [0,1]^n$, using the *same re-uploading structure* as Problem 2 lifted to n inputs. You will (i) define and minimize explicit MSE objectives for both models, (ii) specify the encoding and layer structure, (iii) measure how performance changes with the number of layers L, and (iv) explore where classical simulation remains feasible and where it becomes intractable. Optionally, you may run a small-n instance on an IBM NISQ backend and comment on its performance.

Target functions (choose one or do both).

(T1) **Additive sine:**

$$y(x) \;=\; \sin\big(2\pi\, w^\top x\big), \quad x \in [0,1]^n,$$

with fixed $w \in \mathbb{R}^n$ (e.g. $w = \frac{1}{\sqrt{n}}\mathbf{1}$).

(T2) **Multiplicative sine (higher-order interactions):**

$$y(x) \;=\; \prod_{k=1}^{n} \sin\big(2\pi\, x_k\big).$$

Dataset. Sample N_{train} points $x_i \sim \text{Unif}([0,1]^n)$ and compute labels y_i from (T1) or (T2). Prepare a separate test set (a dense grid for small n, random points otherwise) to report generalization.

Classical model (baseline MLP)

Let $\hat{y}_{\text{MLP}}(x;\boldsymbol{w})$ be a fully connected network with n inputs, L hidden layers (width ≥ 16 recommended), a smooth activation (e.g. tanh), and a linear scalar output.

Objective.

$$\mathcal{L}_{\text{MLP}} \;=\; \frac{1}{N}\sum_{i=1}^{N} \big(\hat{y}_{\text{MLP}}(x_i;\boldsymbol{w}) - y_i\big)^2.$$

Keep the optimizer (e.g. Adam), learning rate, number of epochs, and batch size fixed across all L for a fair comparison. Record train/test MSE and time per epoch.

Quantum model (QNN)

Encoding (angle encoding lifted to n inputs). Use n qubits with input-local rotations,

$$U_\phi(x) \;=\; \bigotimes_{k=1}^{n} R_y\big(\pi x_k\big).$$

Layer / ansatz. A single **layer** at depth ℓ re-uploads the data, applies trainable rotations, and then entangles:

$$U^{(\ell)}_{\text{layer}}(x;\boldsymbol{\theta}^{(\ell)}) \;=\; E \,\cdot\, \Big(\bigotimes_{k=1}^{n} R_y(\theta^{(\ell)}_{k,1})\, R_z(\theta^{(\ell)}_{k,2})\Big) \,\cdot\, U_\phi(x),$$

where the entangler $E = \prod_{k=1}^{n} \mathrm{CNOT}_{k\to k+1}$ is a ring with $n+1 \equiv 1$ for $n \geq 3$, and a single $\mathrm{CNOT}_{1\to 2}$ for $n = 2$. The depth-L circuit is the ordered product

$$U_L(x;\boldsymbol{\theta}) \;=\; \prod_{\ell=1}^{L} U^{(\ell)}_{\text{layer}}(x;\boldsymbol{\theta}^{(\ell)}), \qquad \text{(rightmost factor, } \ell = 1\text{, acts first).}$$

Readout and model output. Measure Pauli-Z on each qubit and combine the results with a *trainable linear head* in classical post-processing:

$$f_{\mathrm{QNN}}(x;\boldsymbol{\theta},\boldsymbol{\alpha}) = \alpha_0 + \sum_{k=1}^{n} \alpha_k \,\langle Z_k\rangle, \qquad \langle Z_k\rangle = \langle 0|^{\otimes n}\, U_L(x;\boldsymbol{\theta})^\dagger\, Z_k\, U_L(x;\boldsymbol{\theta})\, |0\rangle^{\otimes n}.$$

Objective.

$$\mathcal{L}_{\mathrm{QNN}} \;=\; \frac{1}{N}\sum_{i=1}^{N} \big(f_{\mathrm{QNN}}(x_i;\boldsymbol{\theta},\boldsymbol{\alpha}) - y_i\big)^2.$$

Shot-based evaluation and gradients. Estimate each $\langle Z_k\rangle$ from S shots:

$$\widehat{\langle Z_k\rangle} = \frac{n_0^{(k)} - n_1^{(k)}}{S}, \qquad n_0^{(k)} + n_1^{(k)} = S.$$

For each circuit parameter θ, the parameter-shift rule gives $\partial\langle Z_k\rangle/\partial\theta = \frac{1}{2}\big[\langle Z_k\rangle(\theta+\frac{\pi}{2}) - \langle Z_k\rangle(\theta-\frac{\pi}{2})\big]$, and the chain rule gives

$$\frac{\partial \mathcal{L}_{\mathrm{QNN}}}{\partial\theta} = \frac{2}{N}\sum_{i=1}^{N} \big(f_{\mathrm{QNN}}(x_i) - y_i\big) \sum_{k=1}^{n} \alpha_k \,\frac{\partial\langle Z_k\rangle(x_i)}{\partial\theta}, \qquad \frac{\partial \mathcal{L}_{\mathrm{QNN}}}{\partial\alpha_k} = \frac{2}{N}\sum_{i=1}^{N} \big(f_{\mathrm{QNN}}(x_i) - y_i\big)\langle Z_k\rangle(x_i),$$

with $\langle Z_0\rangle \equiv 1$ for the bias α_0.

Experiments: depth vs. performance

Protocol (common to MLP and QNN). For $L \in \{1, 2, 4, 8\}$:

- keep the optimizer, learning rate, epochs, batch size (and, for the QNN, the number of shots S) fixed across depths;
- initialize from the same distribution for every L;
- record final train/test MSE, loss curves, gradient-norm curves, and time per epoch.

Mini-experiment A (depth helps). Use target (T1) with moderate n. Check whether increasing L from 1 to 2 improves the approximation for both the MLP and the QNN. Explain why additional re-uploading layers enlarge the set of trigonometric functions of the encoded angles that the QNN can represent.

Mini-experiment B (does depth hurt?). Use target (T2) with the same n. Train with $L = 4$ and $L = 8$. Do the deeper QNNs converge more slowly or plateau at a worse MSE? Are their gradient norms smaller (a barren-plateau signature)? Contrast with the MLP, which may keep improving modestly with depth.

Deliverables (for $n \in \{2, 3, 4, 6, 8\}$)

(a) Plots for both the classical and quantum models: fits, loss vs. epochs, and gradient norm vs. epochs for $L \in \{1, 2, 4, 8\}$. Comment on the similarities and differences between the plots.

(b) Tables: train/test MSE and time per epoch for each L, for both models; record S.

(c) Reflections: explain the relation between depth and performance, and how the gradient norm behaves differently for the classical and quantum models as n and L grow.